Sparse Gated Adapters for Efficient Low-Resource Text Classification

A. Example and B. Sample (synthetic paper for testing, not a real publication)

Abstract

Adapter modules make it possible to specialize a pretrained encoder with few trainable parameters, but standard adapters are inserted in every layer regardless of need. We introduce Sparse Gated Adapters (SGA), which attach a learned gate to each adapter and penalize the expected number of active adapters. On five low-resource text classification datasets with 500 labeled examples each, SGA improves average macro-F1 by 3.1 points over standard adapters while training only 0.8% of the model parameters. SGA also reduces training time by 27% and increases inference latency by a factor of only 1.04.

1 Introduction

Labeled data is scarce in many practical text classification settings, such as routing customer support tickets or flagging clauses in contracts. Fine-tuning every parameter of a pretrained encoder is expensive and prone to overfitting when only a few hundred examples are available. Adapter modules, small bottleneck networks inserted between transformer layers, reduce the number of trainable parameters, but they are usually added to every layer. We hypothesize that many of these adapters are redundant.

This paper makes three contributions. First, we propose Sparse Gated Adapters, which learn which layers need an adapter. Second, we show that the method improves macro-F1 on five datasets in the low-resource regime. Third, we analyze which layers the gates select and how the gains depend on the amount of labeled data.

2 Related Work

Adapter modules were introduced as a parameter-efficient alternative to full fine-tuning. Prompt tuning instead learns continuous vectors that are prepended to the input while the encoder stays frozen. Bias-only tuning updates just the bias terms of the encoder. Another line of work factorizes weight updates into low-rank matrices. Sparsity-inducing regularizers based on the L0 norm have been used for network pruning; we borrow the hard concrete relaxation from that literature to make the gates trainable.

3 Method

3.1 Adapter Architecture

Each adapter is a bottleneck network that projects the hidden state of dimension 768 down to a bottleneck dimension of r = 32, applies a GELU nonlinearity, and projects back up to 768 dimensions. The output is added to the hidden state through a residual connection. The adapters are placed after the feed-forward sublayer of each of the 12 transformer layers.

3.2 Sparse Gating

We multiply the output of the adapter at layer l by a scalar gate g in the interval from 0 to 1. The gate is sampled from a hard concrete distribution whose parameters are learned, so that gates can take exactly

zero values. At inference time the gate is replaced by its deterministic estimate, and adapters whose gate equals zero are removed from the network. An L0 penalty with coefficient lambda = 0.01 encourages the expected number of active gates to be small.

3.3 Training Objective

The loss is the cross-entropy of the classifier plus the L0 penalty. The classification head and the layer normalization parameters are trained together with the adapters. In total, SGA trains 0.8% of the parameters of the base model, compared with 2.4% for standard adapters.

4 Experimental Setup

4.1 Datasets

We use five text classification datasets: FoodReviews (sentiment, 3 classes), NewsTopics (topic, 8 classes), SupportTickets (intent, 12 classes), ClauseType (legal clause category, 10 classes), and ForumToxicity (binary toxicity detection). For each dataset we sample 500 labeled training examples and evaluate on the full official test split. All datasets are in English.

4.2 Baselines

We compare against four baselines: full fine-tuning, standard adapters with a bottleneck dimension of 32 in every layer, prompt tuning, and bias-only tuning.

4.3 Implementation Details

All methods use a pretrained encoder with 110 million parameters. We train the adapter-based methods with the AdamW optimizer, a learning rate of 3e-4, and a batch size of 16 for up to 20 epochs; full fine-tuning uses a learning rate of 1e-5. Training stops early if validation macro-F1 does not improve for 3 consecutive epochs. We report the mean over 5 random seeds. All experiments run on a single 24 GB GPU.

5 Results

Table 1 reports macro-F1 averaged over the five datasets. SGA reaches an average macro-F1 of 80.0, which is 3.1 points higher than standard adapters (76.9) and 1.6 points higher than full fine-tuning (78.4). SGA reduces training time by 27% relative to standard adapters, from 41 minutes to 30 minutes per run. Because adapters with zero gates are removed, inference latency increases by a factor of only 1.04 relative to the frozen encoder. The improvement is not statistically significant on ForumToxicity ($p = 0.12$), where all adapter-based methods perform similarly.

Table 1: Average macro-F1 over five datasets (500 training examples each).

Method	Trainable params	Avg. macro-F1
Full fine-tuning	100%	78.4
Standard adapters	2.4%	76.9
Prompt tuning	0.1%	71.2
Bias-only tuning	0.1%	73.5
SGA (ours)	0.8%	80.0

6 Analysis

6.1 Ablations

Training the gates without the L0 penalty yields 77.3 macro-F1. Keeping a fixed random 25% of the adapters yields 77.8. Reducing the bottleneck dimension to r = 16 gives 79.1, while increasing it to r = 64 gives 80.1.

6.2 Which Layers Are Selected

The learned gates keep adapters in the upper layers: layers 7 to 12 retain an active gate in 82% of runs, while layers 1 to 6 retain one in only 19% of runs.

6.3 Effect of Training Set Size

With only 100 labeled examples, SGA reaches 68.2 macro-F1 versus 63.5 for standard adapters, a gain of 4.7 points. With 2,000 examples the gap shrinks to 0.9 points.

7 Limitations

Our study has several limitations. All experiments use English datasets and a single encoder with 110 million parameters, so we cannot say whether the findings hold for multilingual or much larger models. We only evaluate classification tasks and do not study generation. The method is sensitive to the sparsity coefficient lambda: values above 0.05 removed almost all adapters and degraded accuracy.

8 Conclusion

We presented Sparse Gated Adapters, a parameter-efficient method that learns where to place adapters. SGA improves macro-F1 over standard adapters in low-resource classification while training fewer parameters and less time. Future work includes multilingual evaluation and larger encoders.

References

[1] Example, A. Adapter modules for transfer learning. Preprint, 2024.

[2] Sample, B. Hard concrete gates for network pruning. Preprint, 2024.